



Apuntes Programación Java

Curso completo de Programación en Java para 1º
DAM/DAW — módulo 0485, RD 405/2023

Indice

- U01 — Introducción a la algorítmica y pseudocódigo
 -  Objetivo de la unidad
 -  La idea en una frase
 -  Las cinco propiedades de un algoritmo
 -  La idea frente al código
 -  Del algoritmo al pseudocódigo
 -  Diagramas de flujo
 -  Los tres bloques de todo programa
 -  Tu entorno de desarrollo: IntelliJ IDEA
 -  Tu primer proyecto en IntelliJ
 -  Sé el Código
 -  ¿No Hay Preguntas Tontas!
 -  Mini-chequeo
 -  Resumen en 3 frases
 -  Boletines de la unidad
 -  Criterios de evaluación cubiertos (RA1)
 -  ¿Por dónde empiezo?
- Boletín U01 — Inicial
 - Ejercicio 1: ordena la receta
 - Ejercicio 2: ¿cumple las cinco propiedades?
 - Ejercicio 3: traza a mano
 - Ejercicio 4: identifica el bloque
 - Ejercicio 5: completa el pseudocódigo
 - Ejercicio 6: empareja conceptos
 - Ejercicio 7: elige el símbolo
 - Ejercicio 8: caza la ambigüedad
- Boletín U01 — Inicial (Resuelto)
 - Ejercicio 1: ordena la receta
 - Ejercicio 2: ¿cumple las cinco propiedades?
 - Ejercicio 3: traza a mano
 - Ejercicio 4: identifica el bloque

- [Ejercicio 5: completa el pseudocódigo](#)
- [Ejercicio 6: empareja conceptos](#)
- [Ejercicio 7: elige el símbolo](#)
- [Ejercicio 8: caza la ambigüedad](#)
- [Boletín U01 — Intermedio](#)
 - [Ejercicio 1: el mayor de tres](#)
 - [Ejercicio 2: traza anidada](#)
 - [Ejercicio 3: traza con bucle](#)
 - [Ejercicio 4: depura el pseudocódigo](#)
 - [Ejercicio 5: del diagrama al pseudocódigo](#)
 - [Ejercicio 6: clasifica el algoritmo completo](#)
 - [Ejercicio 7: tu primer proyecto](#)
 - [Ejercicio 8: propiedades rotas](#)
- [Boletín U01 — Intermedio \(Resuelto\)](#)
 - [Ejercicio 1: el mayor de tres](#)
 - [Ejercicio 2: traza anidada](#)
 - [Ejercicio 3: traza con bucle](#)
 - [Ejercicio 4: depura el pseudocódigo](#)
 - [Ejercicio 5: del diagrama al pseudocódigo](#)
 - [Ejercicio 6: clasifica el algoritmo completo](#)
 - [Ejercicio 7: tu primer proyecto](#)
 - [Ejercicio 8: propiedades rotas](#)
- [Boletín U01 — Extras](#)

U01 — Introducción a la algorítmica y pseudocódigo

Antes de escribir una sola línea de Java, aprende a pensar como un algoritmo 🚀

 **El mapa del curso:**  **AQUÍ ESTÁS (U01)** →  U02 Sintaxis y tipos →  U03 Control y excepciones →  U04 Métodos y recursividad →  U05 Clases y objetos →  U06 Encapsulación →  U07 Herencia →  U08 Arrays →  U09 Genéricos →  U10 Streams →  U11 Ficheros y regex →  U12 GUI y eventos →  U13 JDBC y persistencia →  U14 Proyecto final

Todo el mundo quiere escribir código el primer día de clase. Es comprensible: mola más que rellenar un diagrama. Pero hay un problema — si no sabes pensar en pasos antes de escribir una sola línea, vas a pasar el curso entero copiando sintaxis sin entender por qué funciona lo que funciona.

Así que esta unidad no toca Java. Ni una línea. Vas a aprender a **pensar como un algoritmo**: a descomponer un problema en pasos precisos, a dibujarlo en un diagrama de flujo, a escribirlo en pseudocódigo, y a reconocer los tres bloques con los que se construye absolutamente cualquier programa que existe o existirá. Cuando llegues a la U02 y veas tu primera línea de Java, ya sabrás exactamente qué le estás pidiendo al ordenador que haga — solo te faltará el idioma para decírselo.

También vas a instalar tu entorno de trabajo para todo el curso: **IntelliJ IDEA**. Que sea la única vez en el curso que instalamos algo sin haber programado todavía es una excepción que merece la pena: sin herramienta no hay curso.

Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar **qué es un algoritmo** y enumerar sus cinco propiedades.
- Distinguir la **idea** de un algoritmo de su **materialización** en un lenguaje concreto.
- Escribir un algoritmo sencillo en **pseudocódigo**, sin depender de la sintaxis de ningún lenguaje.
- Representar un algoritmo mediante un **diagrama de flujo** con la simbología estándar.

- Reconocer los **tres bloques estructurales** de todo programa: secuencia, selección e iteración.
- Instalar y configurar **IntelliJ IDEA** como entorno de desarrollo del curso.
- Crear tu primer **proyecto Java** en IntelliJ, aunque todavía no sepas qué hace el código.

La idea en una frase

Un algoritmo es una secuencia finita, ordenada y sin ambigüedades de pasos que resuelve un problema.

Piensa en la última vez que montaste un mueble de esos con instrucciones en dibujitos. Cada paso está numerado, cada pieza está identificada, y el dibujo no deja lugar a dudas sobre qué tornillo va dónde. Eso es un algoritmo bien escrito. Ahora piensa en una instrucción como “encaja las piezas hasta que quede bien” — eso no es un algoritmo, es una esperanza.

El ordenador es exactamente como ese mueble: no interpreta, no da por hecho, no “entiende la intención”. Ejecuta exactamente lo que le dices, ni una coma más. Por eso el trabajo de programar empieza mucho antes de abrir un editor de código: empieza en pensar el problema con la precisión de un manual de instrucciones.

Antes de programar, piensa

La mayoría de los errores de un programador novato no están en la sintaxis — están en no haber pensado bien el algoritmo antes de escribirlo. Un algoritmo mal pensado en Java sigue estando mal pensado, solo que ahora también da errores de compilación.

Las cinco propiedades de un algoritmo

Para que una lista de pasos merezca llamarse algoritmo, tiene que cumplir cinco condiciones:

1. **Finito:** tiene que terminar en algún momento. Un algoritmo que nunca acaba no es un algoritmo, es un cuelgue.
2. **Preciso:** cada paso está definido sin ambigüedad. Nada de “más o menos” ni “cuando esté listo”.
3. **Con entrada:** puede recibir cero o más datos de partida.
4. **Con salida:** produce al menos un resultado.

5. **Eficaz**: resuelve el problema en un tiempo razonable y de forma correcta, no solo “algún día, quizá”.

Un poco de historia

La palabra *algoritmo* viene de **Al-Juarismi**, un matemático persa del siglo IX que escribió uno de los primeros tratados sobre cómo hacer cálculos siguiendo procedimientos sistemáticos. Más de mil años después, seguimos usando su nombre — deformado— para lo mismo: procedimientos que se pueden seguir paso a paso sin margen de interpretación.

La idea frente al código

Este es el concepto que sostiene toda la unidad, así que no lo pases por encima.

No confundas el algoritmo con el código

El **algoritmo** es la idea: la secuencia de pasos. El **código** es su traducción a un lenguaje concreto. El mismo algoritmo “suma dos números y muestra el resultado” se puede escribir en Java, en Python o en pseudocódigo, y la esencia del algoritmo — los pasos, el orden, la lógica — no cambia en absoluto.

Ejemplo: el algoritmo “calcular el área de un rectángulo” se puede escribir así en pseudocódigo:

```
INICIO
  LEER base
  LEER altura
  area ← base * altura
  ESCRIBIR area
FIN
```

Y dentro de un par de semanas, ese mismo algoritmo, sin cambiar ni un paso de la lógica, será esto en Java:

```
Scanner sc = new Scanner(System.in);
double base = sc.nextDouble();
double altura = sc.nextDouble();
double area = base * altura;
System.out.println(area);
```

Fíjate: cuatro pasos, en el mismo orden, en los dos casos. Lo único que cambia es el idioma. Por eso en esta unidad trabajamos en pseudocódigo — para que la lógica no se te mezcle con la sintaxis todavía.

Del algoritmo al pseudocódigo

El **pseudocódigo** es una forma de escribir algoritmos con una estructura parecida a la de un lenguaje de programación, pero sin las reglas estrictas de sintaxis de ninguno en concreto. Es el punto intermedio entre “explicarlo con palabras” y “escribirlo en Java”.

No hay una única norma de pseudocódigo — cada libro tiene la suya — pero en este curso usaremos estas convenciones:

Convención	Significado
INICIO / FIN	Marcan el principio y el final del algoritmo
LEER <code>variable</code>	Pide un dato de entrada
ESCRIBIR <code>expresión</code>	Muestra un resultado
<code>variable</code> ← <code>expresión</code>	Asignación: guarda el resultado de la expresión en la variable
SI <code>condición</code> ENTONCES ... SINO ... FINSI	Bifurcación: decide entre dos caminos
MIENTRAS <code>condición</code> HACER ... FINMIENTRAS	Repetición: repite mientras se cumpla la condición

Ejemplo — el algoritmo que decide si un número es par o impar:

```
INICIO
  LEER numero
  SI numero MOD 2 = 0 ENTONCES
    ESCRIBIR "Es par"
  SINO
    ESCRIBIR "Es impar"
  FINSI
FIN
```

Nadie te va a examinar de escribir MOD en vez de %, ni de si pones ENTONCES o ->. Lo que importa es que **cualquier persona que lea tu pseudocódigo pueda seguirlo paso a paso sin dudar**, igual que las instrucciones del mueble.

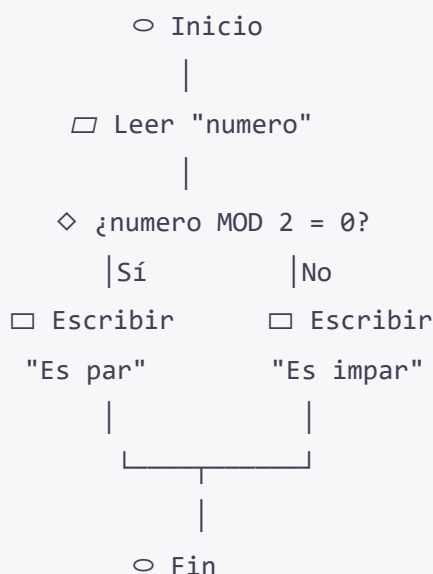
◆ Diagramas de flujo

Un **diagrama de flujo** es la versión dibujada del pseudocódigo: la misma lógica, pero representada con figuras geométricas conectadas por flechas que marcan el orden de ejecución. Para muchas personas, ver la lógica en forma de dibujo hace mucho más fácil detectar dónde falla un razonamiento.

La simbología estándar que usaremos:

Símbolo	Forma	Significado
Óvalo	○	Inicio o fin del algoritmo
Rectángulo	□	Proceso o acción (un cálculo, una asignación)
Paralelogramo	▱	Entrada o salida de datos
Rombo	◇	Decisión (una pregunta con respuesta sí/no)
Flecha	→	Indica el orden de ejecución

El algoritmo del número par/impar de antes, en diagrama de flujo, se leería así (de arriba abajo):



Truco para no perderte

Sigue siempre la flecha con el dedo, nunca “a ojo”. El 90 % de los errores de lógica de un principiante se detectan solos en cuanto trazas el diagrama paso a paso en vez de darlo por bueno de un vistazo.



Los tres bloques de todo programa

Aquí está el secreto que hace que la programación sea aprendible: **por complicado que parezca un programa, siempre está construido combinando solo tres bloques estructurales**. Nada más. Todo lo que verás en este curso — desde un “Hola Mundo” hasta un proyecto con base de datos — es una combinación de estos tres bloques, unos dentro de otros:

1. **Secuencia**: los pasos se ejecutan uno detrás de otro, en el orden en que están escritos. Es el bloque por defecto — si no dices nada, el programa es secuencial.
2. **Selección** (o decisión): el programa elige entre dos o más caminos según se cumpla o no una condición. Es el SI...ENTONCES...SINO de pseudocódigo, el rombo del diagrama de flujo.
3. **Iteración** (o repetición): un bloque de pasos se repite mientras se cumpla una condición. Es el MIENTRAS...HACER de pseudocódigo.

SECUENCIA	SELECCIÓN	ITERACIÓN
	◇ ¿condición?	◇ ¿condición? ←
□	Sí No	Sí
	□ □	□ —————
□		No
		(sale del bucle)

En la U03 vas a escribir estos tres bloques en Java de verdad (if, else, while, for). Por ahora, lo que importa es que los reconozcas *como idea*: cuando leas cualquier algoritmo, pregúntate siempre “¿esto es una secuencia, una decisión o una repetición?”. Si sabes responder esa pregunta, sabes leer código antes de saber escribirlo.



Tu entorno de desarrollo: IntelliJ IDEA

Todo el curso se apoya en un único IDE: **IntelliJ IDEA**. No vamos a usar Eclipse, ni VS Code, ni Notepad++ con la terminal al lado — cualquier captura, atajo de teclado o menú que veas en los apuntes está pensado para IntelliJ, así que instalarlo ahora te va a ahorrar confusiones más adelante.

Instalación:

1. Descarga **IntelliJ IDEA Community Edition** (gratuita, de código abierto) desde el sitio oficial de JetBrains.
2. Ejecuta el instalador con las opciones por defecto.
3. IntelliJ puede descargarte un JDK (Java Development Kit) automáticamente la primera vez que crees un proyecto — elige siempre la última versión LTS disponible.
4. Al abrir IntelliJ por primera vez, verás la pantalla de bienvenida con la opción **New Project**.

¿Community o Ultimate?

La edición **Community** es gratuita y tiene todo lo que necesitas para este curso: soporte de Java, depurador, gestión de proyectos. La edición **Ultimate** añade herramientas para desarrollo web y empresarial que no usaremos hasta unidades muy avanzadas — no hace falta pagarla ahora.

Tu primer proyecto en IntelliJ

Aunque todavía no vayamos a escribir lógica propia, familiarizarte con la creación de un proyecto ahora te ahorra fricción en la U02, cuando sí toque programar de verdad.

1. En la pantalla de bienvenida, pulsa **New Project**.
2. Elige **Java** como tipo de proyecto y comprueba que el **JDK** seleccionado es una versión LTS.
3. Ponle un nombre al proyecto — por ejemplo, CursoProgramacion — y elige dónde guardarlo.
4. Marca la opción de crear un ejemplo de código si IntelliJ te la ofrece; genera una clase con un main ya escrito.
5. Pulsa **Create**. IntelliJ generará la estructura del proyecto: una carpeta `src` con el código fuente y los ficheros de configuración del proyecto.

6. Verás el árbol del proyecto en el panel izquierdo y el editor de código en el centro. Ese va a ser tu escritorio de trabajo durante las próximas trece unidades.

No canceles la sincronización

Si IntelliJ te pide indexar el proyecto o descargar componentes la primera vez, déjalo terminar. Cancelarlo a mitad suele dejar el proyecto en un estado raro que cuesta más arreglar que esperar dos minutos.

Un **proyecto** en IntelliJ (y en cualquier IDE serio) no es solo tu fichero de código: es la carpeta completa con la configuración, las dependencias y los metadatos que el IDE necesita para compilar y ejecutar tu programa con un solo clic. Cuando en la U02 crees tu primera clase Java de verdad, la crearás dentro de este proyecto.

Sé el Código

Ponte en la piel del ordenador: no interpretes, no completes huecos, no asumas nada que no esté escrito.

Ejercicio: el siguiente pseudocódigo calcula si un alumno aprueba una asignatura. Traza su ejecución a mano para las notas $\text{nota} = 4.5$ y $\text{nota} = 6.0$, escribiendo qué muestra ESCRIBIR en cada caso.

```
INICIO
  LEER nota
  SI nota >= 5 ENTONCES
    ESCRIBIR "Aprobado"
  SINO
    ESCRIBIR "Suspenso"
  FINSI
  ESCRIBIR "Nota registrada: " + nota
FIN
```

>  **Solución**

¡No Hay Preguntas Tontas!

¿Puedo saltarme el pseudocódigo y pensar directamente en Java? Puedes, y en algún momento del curso lo harás sin darte cuenta. Pero mientras estás aprendiendo, mezclar “pensar la lógica” con “recordar la sintaxis” a la vez es la forma más rápida de bloquearte delante de un ejercicio en blanco. El pseudocódigo separa los dos problemas.

¿Todos los diagramas de flujo se leen de arriba abajo? Por convenio sí, aunque algunas herramientas los dibujan de izquierda a derecha. Lo importante no es la dirección, sino seguir siempre las flechas y no dar ningún salto “porque se entiende”.

¿Por qué instalamos IntelliJ si todavía no sabemos programar? Porque el entorno de trabajo no depende de saber programar — depende de tener algo instalado para cuando sí toque. Es como aprender dónde están los fogones antes de aprender a cocinar: no cocinas mejor por saberlo, pero cocinas antes.

¿Qué pasa si mi algoritmo no cumple alguna de las cinco propiedades? Que no es un algoritmo todavía, es un boceto. No pasa nada — es lo normal la primera vez que intentas resolver un problema. Revísalo hasta que las cumpla las cinco: ahí es cuando está listo para convertirse en código.

Mini-chequeo

Ponte a prueba en 30 segundos:

1. ¿Cuáles son las cinco propiedades de un algoritmo?
2. ¿Qué diferencia hay entre el algoritmo y el código que lo implementa?
3. ¿Qué símbolo del diagrama de flujo representa una decisión?
4. Nombra los tres bloques estructurales de cualquier programa.

>  Respuestas

Resumen en 3 frases

1. Un **algoritmo** es una secuencia finita, precisa y sin ambigüedades de pasos que resuelve un problema, y debe cumplir cinco propiedades: finito, preciso, con entrada, con salida y eficaz.
2. El **pseudocódigo** y los **diagramas de flujo** permiten expresar la lógica de un algoritmo sin depender todavía de la sintaxis de ningún lenguaje concreto.

3. Cualquier programa, por complejo que sea, se construye combinando solo tres bloques: **secuencia, selección e iteración.**

Vocabulario rápido

Término	Idea general
Algoritmo	Secuencia finita y precisa de pasos que resuelve un problema
Pseudocódigo	Escritura de un algoritmo sin la sintaxis estricta de un lenguaje concreto
Diagrama de flujo	Representación gráfica de un algoritmo mediante símbolos y flechas
Secuencia	Bloque en el que los pasos se ejecutan uno detrás de otro
Selección	Bloque que elige entre caminos según una condición
Iteración	Bloque que repite pasos mientras se cumple una condición
IDE	Entorno integrado de desarrollo (en este curso, IntelliJ IDEA)
Proyecto	Carpeta con el código y la configuración que el IDE necesita para compilarlo y ejecutarlo

Boletines de la unidad

Practica con los pares del curso: empieza siempre por el resuelto para ver el estilo, y luego inténtalo por tu cuenta.

✓ Inicial resuelto

● Inicial por resolver

👉 Intermedio resuelto

★ Intermedio por resolver

🔥 Extras

✓ Criterios de evaluación cubiertos (RA1)

RA1: Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.

CE	Criterio	Dónde se cubre
a)	Bloques que componen la estructura de un programa	✓ “Los tres bloques de todo programa”
b)	Proyectos de desarrollo de aplicaciones	✓ “Tu primer proyecto en IntelliJ”
c)	Entornos integrados de desarrollo	✓ “Tu entorno de desarrollo: IntelliJ IDEA”

✦ Los criterios d) a i) de RA1 (variables, constantes, operadores, conversiones de tipo, comentarios) se cubren en la U02, cuando estos bloques se traducen por fin a sintaxis Java real.

■ ¿Por dónde empiezo?

- ¿Cero conocimientos de programación? → Lee la unidad entera en orden, no te saltes nada.
- ¿Ya sabes qué es un algoritmo de otra asignatura? → Ve directa a “Los tres bloques de todo programa” y a la instalación de IntelliJ.
- ¿Ya tienes IntelliJ instalado? → Sáltate la instalación y repasa el pseudocódigo antes de los boletines.
- ¿Vienes a repasar? → Haz el Mini-chequeo y después los [boletines](#).

▶▶ **Al acabar la unidad, continúa en U02 - Primeros pasos en Java: sintaxis y tipos de datos.**

Boletín U01 — Inicial



Boletín U01 — Inicial

Sin soluciones. Sin prisas. Con papel y lápiz — todavía no hace falta abrir IntelliJ.

Ejercicio 1: ordena la receta

Estos pasos para preparar un café con leche están desordenados. Numéralos del 1 al 5 para formar un algoritmo válido.

- Servir el café en la taza.
- Añadir la leche caliente.
- Calentar la leche.
- Poner la cafetera en el fuego.
- Esperar a que salga el café.

Ejercicio 2: ¿cumple las cinco propiedades?

Para cada “algoritmo”, indica si cumple las cinco propiedades (finito, preciso, con entrada, con salida, eficaz) y, si no, cuál falla:

- a) “Repite: suma 1 a un contador. No pares nunca.” b) “Lee un número. Si es mayor que 10, escribe ‘grande’. Si no, escribe ‘pequeño’.” c) “Cocina la pasta el tiempo que veas.” d) “Lee dos números. Calcula y escribe su suma.”

Ejercicio 3: traza a mano

Traza este pseudocódigo para `edad = 15` y para `edad = 20`. Escribe exactamente lo que muestra `ESCRIBIR` en cada caso.

```
INICIO
  LEER edad
  SI edad >= 18 ENTONCES
    ESCRIBIR "Eres mayor de edad"
  SINO
    ESCRIBIR "Eres menor de edad"
  FINSI
FIN
```

Ejercicio 4: identifica el bloque

Para cada fragmento de pseudocódigo, indica si es **secuencia**, **selección** o **iteración**:

a)

```
LEER a
LEER b
ESCRIBIR a + b
```

b)

```
MIENTRAS contador < 10 HACER
  contador ← contador + 1
FINMIENTRAS
```

c)

```
SI nota >= 5 ENTONCES
  ESCRIBIR "Aprobado"
FINSI
```

Ejercicio 5: completa el pseudocódigo

Falta una línea en este algoritmo que debería mostrar si un número es positivo o negativo. Añádela donde corresponda.


```
INICIO
  LEER numero
  SI numero >= 0 ENTONCES
    ESCRIBIR "Positivo"
  SINO

  FINSI
FIN
```

Ejercicio 6: empareja conceptos

Relaciona cada concepto de la izquierda con su definición de la derecha:

Concepto	Definición
1. Algoritmo	A. Representación gráfica con símbolos y flechas
2. Pseudocódigo	B. Secuencia finita y precisa de pasos que resuelve un problema
3. Diagrama de flujo	C. Carpeta con el código y la configuración que usa el IDE
4. Proyecto	D. Escritura de un algoritmo sin sintaxis de un lenguaje concreto

Escribe las respuestas como “1→B, 2→D, ...”.

Ejercicio 7: elige el símbolo

¿Qué símbolo de diagrama de flujo usarías para representar cada uno de estos pasos?

a) “Leer el nombre del usuario.” b) “¿La contraseña es correcta?” c) “Calcular el total sumando precio más impuestos.” d) “Inicio del algoritmo.”

Ejercicio 8: caza la ambigüedad

Cada uno de estos pasos es ambiguo y no serviría dentro de un algoritmo preciso. Reescribe cada uno para que no deje lugar a interpretación:

a) “Añade sal al gusto.” b) “Espera un rato.” c) “Si el número es grande, avisa.” d) “Repite varias veces.”

Boletín U01 — Inicial (Resuelto)



Boletín U01 — Inicial (Resuelto)

Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: ordena la receta

>  Solución

Ejercicio 2: ¿cumple las cinco propiedades?

>  Solución

Ejercicio 3: traza a mano

>  Solución

Ejercicio 4: identifica el bloque

>  Solución

Ejercicio 5: completa el pseudocódigo

>  Solución

Ejercicio 6: empareja conceptos

>  Solución

Ejercicio 7: elige el símbolo

>  Solución

Ejercicio 8: caza la ambigüedad

>  Solución

Boletín U01 — Intermedio



Boletín U01 — Intermedio

Sin soluciones. Aquí ya toca pensar el algoritmo desde cero, no solo leerlo.

Ejercicio 1: el mayor de tres

Escribe en pseudocódigo un algoritmo que lea tres números y escriba cuál es el mayor de los tres.

Ejercicio 2: traza anidada

Traza este pseudocódigo para $n1 = 7, n2 = 7$ y para $n1 = 3, n2 = 9$. Escribe la salida exacta en cada caso.

```
INICIO
  LEER n1
  LEER n2
  SI n1 = n2 ENTONCES
    ESCRIBIR "Son iguales"
  SINO
    SI n1 > n2 ENTONCES
      ESCRIBIR "n1 es mayor"
    SINO
      ESCRIBIR "n2 es mayor"
  FINSI
FINSI
FIN
```

Ejercicio 3: traza con bucle

Traza este pseudocódigo y escribe todo lo que se muestra por pantalla, línea a línea.

INICIO

contador $\leftarrow$ 1

MIENTRAS contador $\leq$ 5 HACER

 ESCRIBIR contador

 contador $\leftarrow$ contador + 1

FINMIENTRAS

ESCRIBIR "Fin del bucle"

FIN

Ejercicio 4: depura el pseudocódigo

Este algoritmo debería sumar los números del 1 al 10, pero tiene un error de lógica que lo deja en bucle infinito. Encuéntralo y corrígelo.

INICIO

suma $\leftarrow$ 0

contador $\leftarrow$ 1

MIENTRAS contador $\leq$ 10 HACER

 suma $\leftarrow$ suma + contador

FINMIENTRAS

ESCRIBIR suma

FIN

Ejercicio 5: del diagrama al pseudocódigo

Un diagrama de flujo tiene esta estructura: empieza, pide un número, decide si es par (si el resto de dividir entre 2 es 0), si es par escribe “Par”, si no escribe “Impar”, y termina. Escribe el pseudocódigo correspondiente.

Ejercicio 6: clasifica el algoritmo completo

Para el pseudocódigo del ejercicio 3 (el del bucle del 1 al 5), identifica qué líneas concretas corresponden a cada uno de los tres bloques estructurales: secuencia, selección e iteración. (Pista: puede que falte alguno de los tres bloques — si es así, dilo.)

Ejercicio 7: tu primer proyecto

Abre IntelliJ IDEA y crea un nuevo proyecto Java llamado PracticaU01. Una vez creado, escribe qué carpetas y ficheros ha generado el IDE automáticamente y para qué sirve cada uno, a partir de lo que veas en el panel del proyecto.

Ejercicio 8: propiedades rotas

Cada uno de estos “algoritmos” incumple una o más de las cinco propiedades. Para cada uno, indica qué propiedad(es) falla(n) y reescríbelo en pseudocódigo correcto:

a) “Mientras el usuario no se canse, pide un número y súmalo al total.” b) “Calcula la raíz cuadrada de un número. No hace falta pedir el número, ya se sabe cuál es.” c) “Lee un número. Si es primo, tarda lo que tarde en decírtelo.”

Boletín U01 — Intermedio (Resuelto)



Boletín U01 — Intermedio (Resuelto)

Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: el mayor de tres

>  Solución

Ejercicio 2: traza anidada

>  Solución

Ejercicio 3: traza con bucle

>  Solución

Ejercicio 4: depura el pseudocódigo

>  Solución

Ejercicio 5: del diagrama al pseudocódigo

>  Solución

Ejercicio 6: clasifica el algoritmo completo

>  Solución

Ejercicio 7: tu primer proyecto

>  Solución

Ejercicio 8: propiedades rotas

>  Solución

Boletín U01 — Extras



Boletín U01 — Extras

★ **Extras** — CodeWars y AceptaElReto cogen ritmo a partir de la U03, cuando ya sepas escribir estructuras de control en Java de verdad. Sin sintaxis todavía, no hay katas que resolver con garantías.

Mientras tanto, la mejor forma de ampliar es esta: coge cualquier proceso cotidiano (hacer la maleta, planificar una ruta, ordenar tu lista de reproducción) y escríbelo en pseudocódigo completo, con sus cinco propiedades bien cubiertas. Si te sobra tiempo, dibuja también el diagrama de flujo.